Sequence Diagrams: Overall System and Each Layer

VIB Internal Knowledge Assistant (Slack RAG bot)

COS40005 Computing Technology Project A · Swinburne University of Technology
Version for team review · 31 October 2026

Status: draft for team discussion. Items marked 'Design assumption' or 'Proposal' are not yet agreed with the client.

Contents

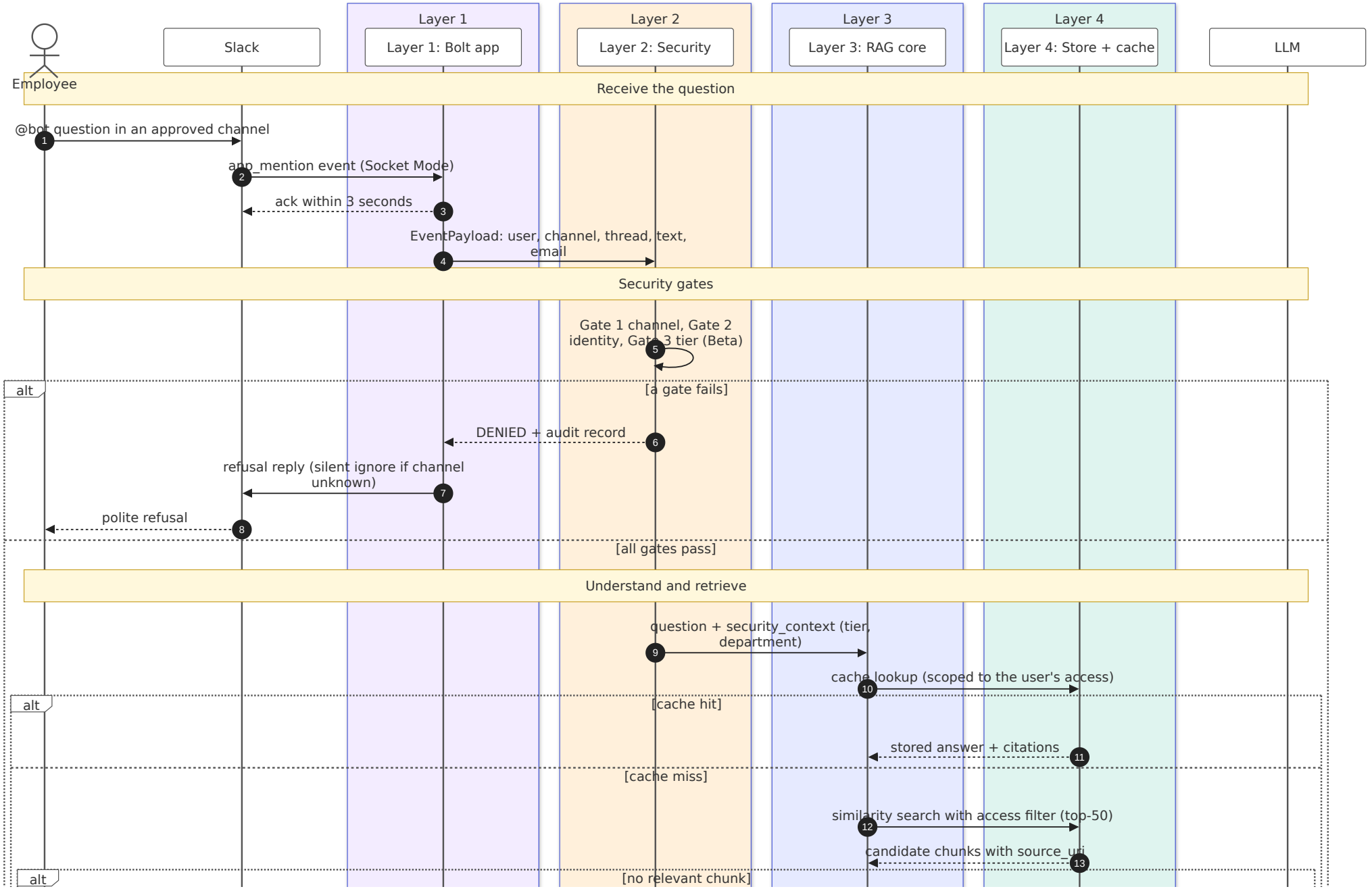
1. Overall query flow (end to end) [Overview]
2. Overall ingestion flow (Confluence to knowledge base) [Layer 4]
3. Layer 1: Presentation (Slack app interface) [Layer 1]
4. Layer 2: Security and authentication [Layer 2]
5. Layer 3: RAG core and business logic [Layer 3]
6. Layer 4: Ingestion in detail (offline) [Layer 4]
7. Layer 4: Query-time retrieval and cache [Layer 4]
8. Design assumptions to confirm with the team

How to read the diagrams

- Each vertical line is one participant. Time runs from top to bottom.
- Solid arrow = a request or call. Dashed arrow = a reply. An arrow that loops back to the same participant = work done inside it.
- Numbers in circles are step numbers. The tables after each diagram use the same numbers.
- Frames: alt = either branch happens (if / else), opt = optional, only if the condition is true, loop = repeated for each item.
- Yellow bars = phases of the flow. Coloured background groups = the four layers (L1 purple, L2 orange, L3 blue, L4 green).
- (Beta) = planned for the Beta release, not required for Alpha. 'Design assumption' = added by the diagram author and needs team confirmation.

Numbering note: step numbers restart in every diagram. The earlier single query diagram (30 steps) is replaced by diagram 1 plus the layer diagrams 3 to 7.

Diagram 1: Overall query flow (end to end) (1/2)



Overview

Diagram 1: Overall query flow (end to end) (2/2)

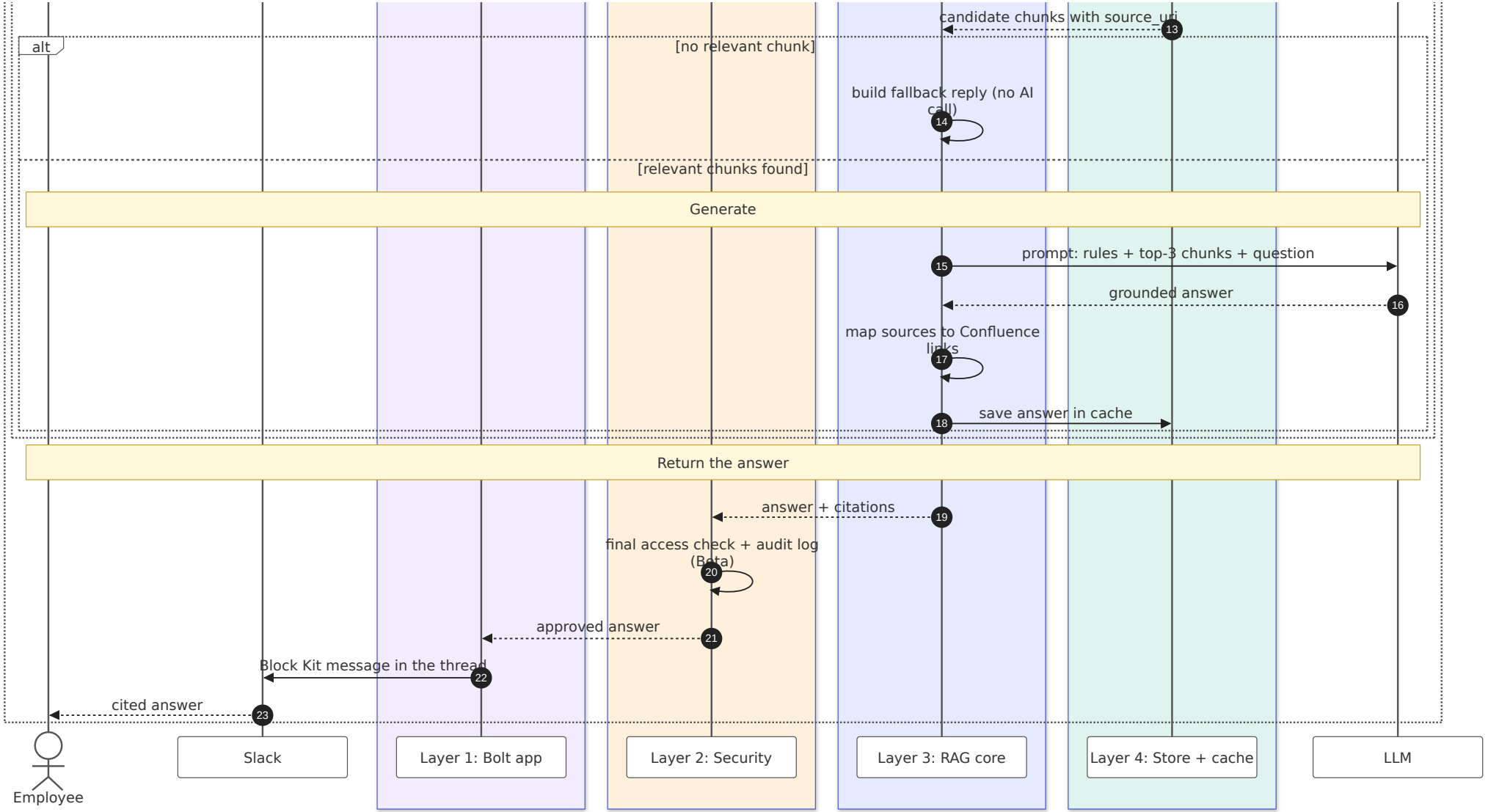


Diagram 1: Overall query flow (end to end)

Overview

Purpose

One question travels through all four layers and comes back as a cited answer. Each layer is shown as a single participant, so you see the whole journey on one page. Diagrams 3 to 7 open each layer up and show what happens inside.

Participants

Participant	Role
Employee	Asks the question in Slack
Slack	Chat platform (external)
Layer 1: Bolt app	Receives events, shows answers
Layer 2: Security	Gates, access context, audit
Layer 3: RAG core	Understands, retrieves, generates
Layer 4: Store + cache	Searchable knowledge and answer cache
LLM	AI model (Ollama or Bedrock)

Step by step

#	From → To	What happens	Why / note
Receive the question			
1	Employee → Slack	@bot question in an approved channel	The bot reacts only when mentioned. It does not read other messages.
2	Slack → Layer 1: Bolt app	app_mention event (Socket Mode)	Slack pushes the event over the WebSocket the bot opened, so no public URL is needed.
3	Layer 1: Bolt app → Slack	↩ ack within 3 seconds	Slack treats a slower reply as a failure and sends the event again.
4	Layer 1: Bolt app → Layer 2: Security	EventPayload: user, channel, thread, text, email	Layer 1 only collects facts. Every access decision is made in Layer 2.
Security gates			
5	Layer 2: Security ⟲	Gate 1 channel, Gate 2 identity, Gate 3 tier (Beta)	Checks run in a fixed order. The cheapest check runs first.
alt: a gate fails			
6	Layer 2: Security → Layer 1: Bolt app	↩ DENIED + audit record	Nothing is retrieved and nothing is sent to the AI.
7	Layer 1: Bolt app → Slack	refusal reply (silent ignore if channel unknown)	An unknown channel is ignored without any reply, so the bot does not reveal itself. A refused person gets a short polite message. (Design assumption.)
8	Slack → Employee	↩ polite refusal	
else: all gates pass			
Understand and retrieve			
9	Layer 2: Security → Layer 3: RAG core	question + security_context (tier, department)	The security context travels with the question and drives the search filter.

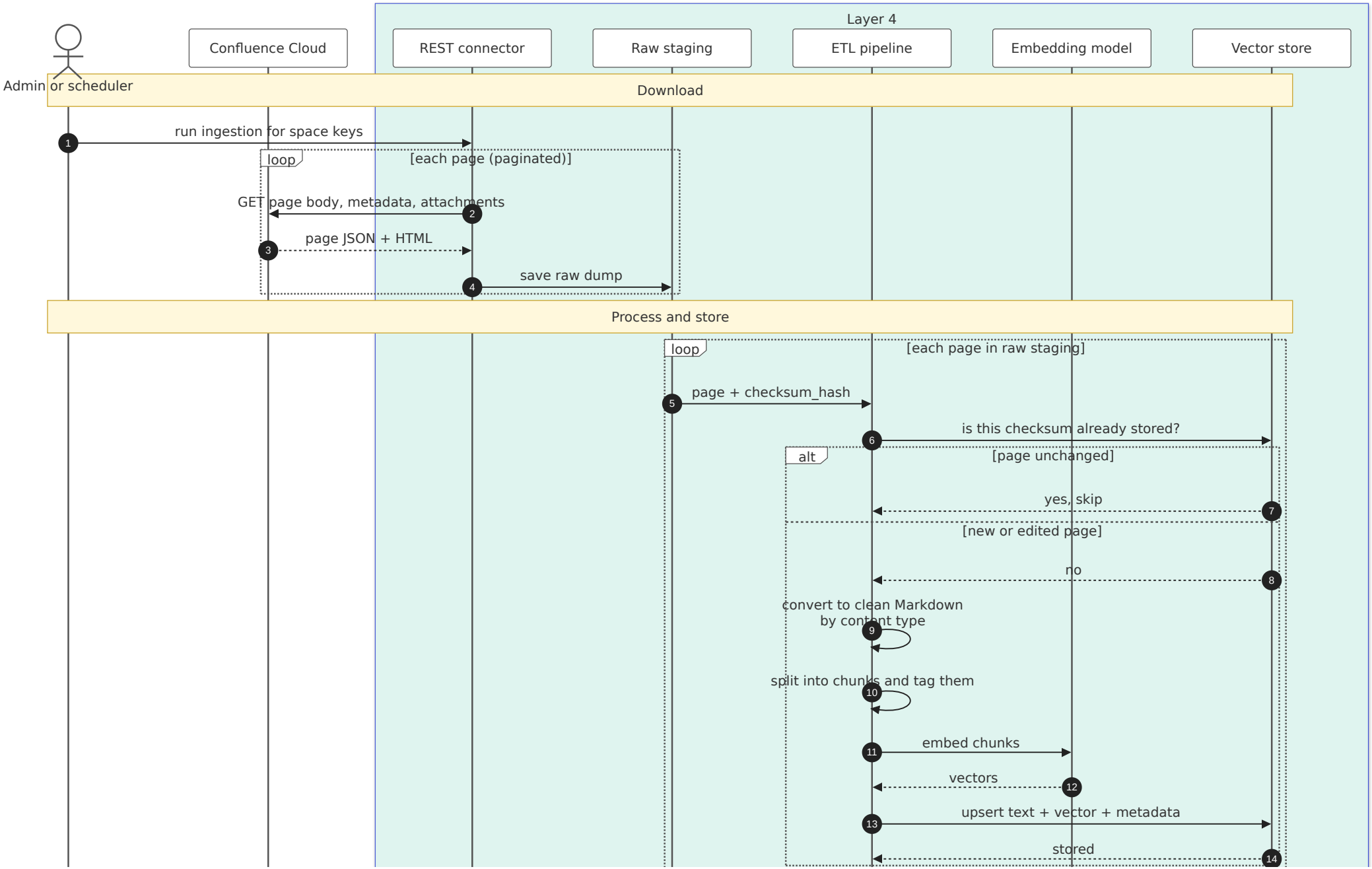
#	From → To	What happens	Why / note
10	Layer 3: RAG core → Layer 4: Store + cache	cache lookup (scoped to the user's access)	A cached answer is reused only for people with the same access scope. Otherwise a manager's answer could leak to an employee. (Design assumption.)
alt: cache hit			
11	Layer 4: Store + cache → Layer 3: RAG core	↩ stored answer + citations	The search, the re-ranking and the AI call are all skipped.
else: cache miss			
12	Layer 3: RAG core → Layer 4: Store + cache	similarity search with access filter (top-50)	The filter is part of the database query, so forbidden text is never returned.
13	Layer 4: Store + cache → Layer 3: RAG core	↩ candidate chunks with source_uri	
alt: no relevant chunk			
14	Layer 3: RAG core ⟲	build fallback reply (no AI call)	Saying 'not found' is safer than guessing.
else: relevant chunks found			
Generate			
15	Layer 3: RAG core → LLM	prompt: rules + top-3 chunks + question	Rules: answer only from the context, cite every source, reply in the language of the question.
16	LLM → Layer 3: RAG core	↩ grounded answer	
17	Layer 3: RAG core ⟲	map sources to Confluence links	
18	Layer 3: RAG core → Layer 4: Store + cache	save answer in cache	Only answers that have sources are cached. Fallback replies are not.
Return the answer			
19	Layer 3: RAG core → Layer 2: Security	↩ answer + citations	Also used for cached answers and for the fallback reply (which has no citations).
20	Layer 2: Security ⟲	final access check + audit log (Beta)	Every cited document must be inside the asker's access. One JSON log line is written per request.
21	Layer 2: Security → Layer 1: Bolt app	↩ approved answer	
22	Layer 1: Bolt app → Slack	Block Kit message in the thread	The answer card includes a button for each source.
23	Slack → Employee	↩ cited answer	

Key points

- Every question passes three security gates before anything is searched. There is no shortcut from Slack to the AI.
- The access filter is part of the database query, so text the person may not read is never returned and never reaches the AI.
- There are three early exits: refused (steps 6 to 8), cache hit (step 11) and nothing found (step 14). Only the last branch of the search calls the AI model.
- Items marked Beta (tier check, audit log) can be switched off without breaking the Alpha flow.

Layer 4

Diagram 2: Overall ingestion flow (Confluence to knowledge base) (1/2)



Layer 4

Diagram 2: Overall ingestion flow (Confluence to knowledge base) (2/2)

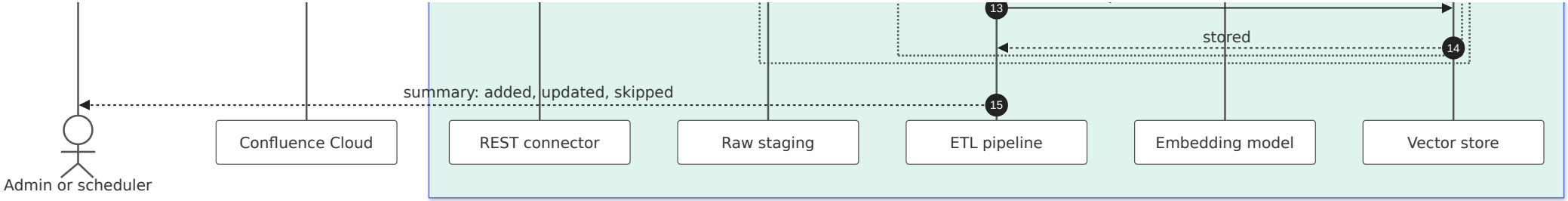


Diagram 2: Overall ingestion flow (Confluence to knowledge base)

Layer 4

Purpose

This diagram shows what happens before anyone asks a question. Confluence pages are downloaded, cleaned, cut into pieces, tagged with who may see them, turned into vectors and stored. It runs offline, started by an admin or a schedule. Diagram 6 shows the same flow in more detail.

Participants

Participant	Role
Admin or scheduler	Starts a run
Confluence Cloud	Source of the pages (external)
REST connector	Downloads pages through the API
Raw staging	Keeps the downloaded originals
ETL pipeline	Parse, chunk and tag
Embedding model	Turns text into vectors
Vector store	PostgreSQL with pgvector

Step by step

#	From → To	What happens	Why / note
Download			
1	Admin or scheduler → REST connector	run ingestion for space keys	Alpha uses one mock space, VIB-Mock-Knowledge. A run is started by hand or by a schedule.
loop: each page (paginated)			
2	REST connector → Confluence Cloud	GET page body, metadata, attachments	
3	Confluence Cloud → REST connector	↩ page JSON + HTML	
4	REST connector → Raw staging	save raw dump	Lets the parsers be re-run without calling Confluence again.
Process and store			
loop: each page in raw staging			
5	Raw staging → ETL pipeline	page + checksum_hash	The checksum is an MD5 of the page content. It detects changes and is not used for security.
6	ETL pipeline → Vector store	is this checksum already stored?	
alt: page unchanged			
7	Vector store → ETL pipeline	↩ yes, skip	
else: new or edited page			
8	Vector store → ETL pipeline	↩ no	

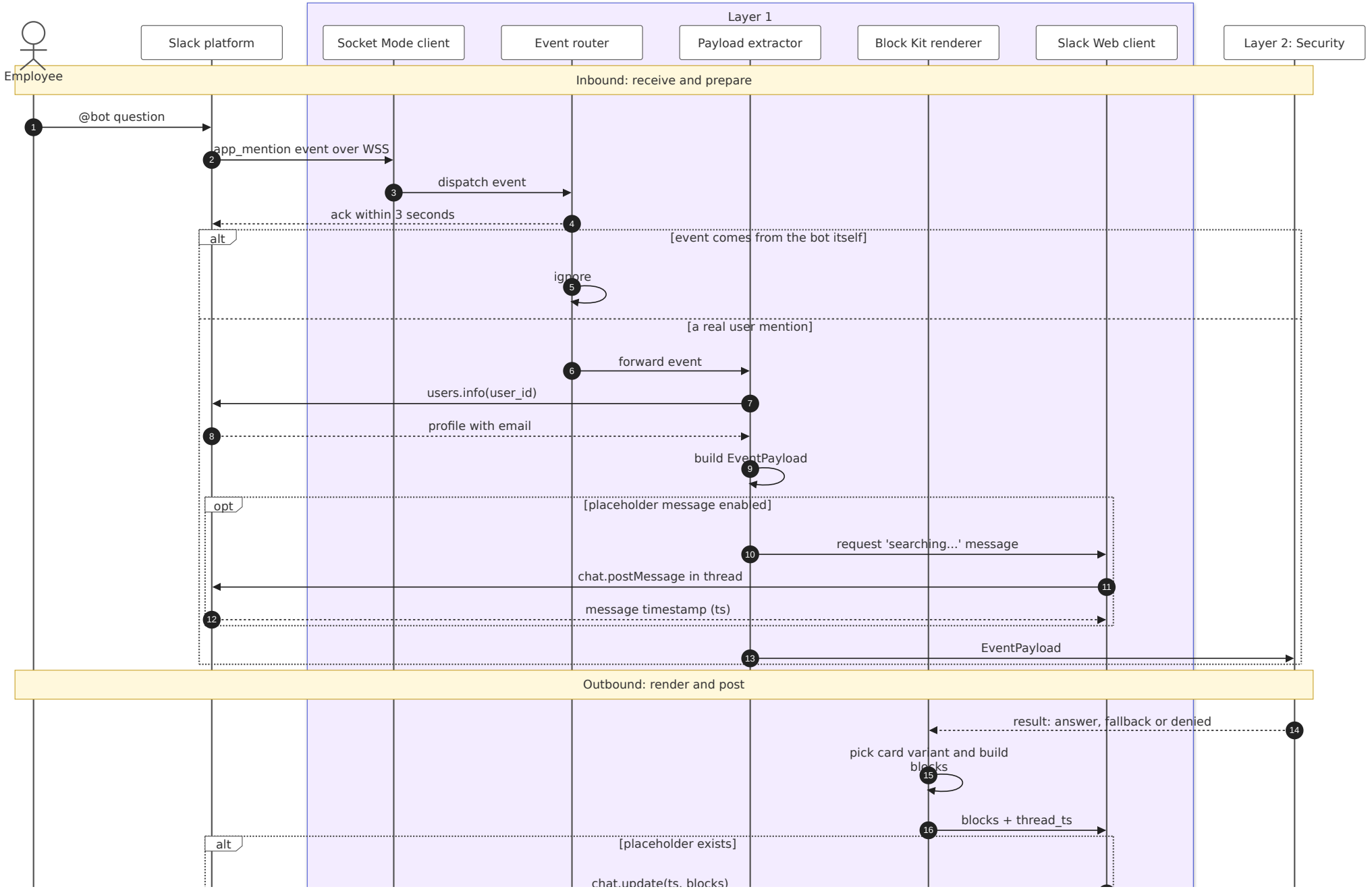
#	From → To	What happens	Why / note
9	ETL pipeline ⟲	convert to clean Markdown by content type	Alpha: wiki HTML, tables, images. Beta: Word, PDF, Excel, OCR.
10	ETL pipeline ⟲	split into chunks and tag them	512 tokens per chunk with 50 overlap. Tags: space, department, file type, access level, source link, checksum.
11	ETL pipeline → Embedding model	embed chunks	
12	Embedding model → ETL pipeline	↩ vectors	
13	ETL pipeline → Vector store	upsert text + vector + metadata	
14	Vector store → ETL pipeline	↩ stored	
15	ETL pipeline → Admin or scheduler	↩ summary: added, updated, skipped	Keep this in the log to see which run had a problem.

Key points

- Raw pages are saved first, so the parsers can be re-run without calling Confluence again.
- The checksum of each page decides whether it is processed again. Unchanged pages are skipped, so a second run is fast.
- The access label is stamped on each piece here. If this step is wrong, every later permission filter is wrong, so it needs the most testing.
- The same embedding model must be used here and at question time, otherwise the vectors cannot be compared.

Layer 1

Diagram 3: Layer 1: Presentation (Slack app interface) (1/2)



Layer 1

Diagram 3: Layer 1: Presentation (Slack app interface) (2/2)

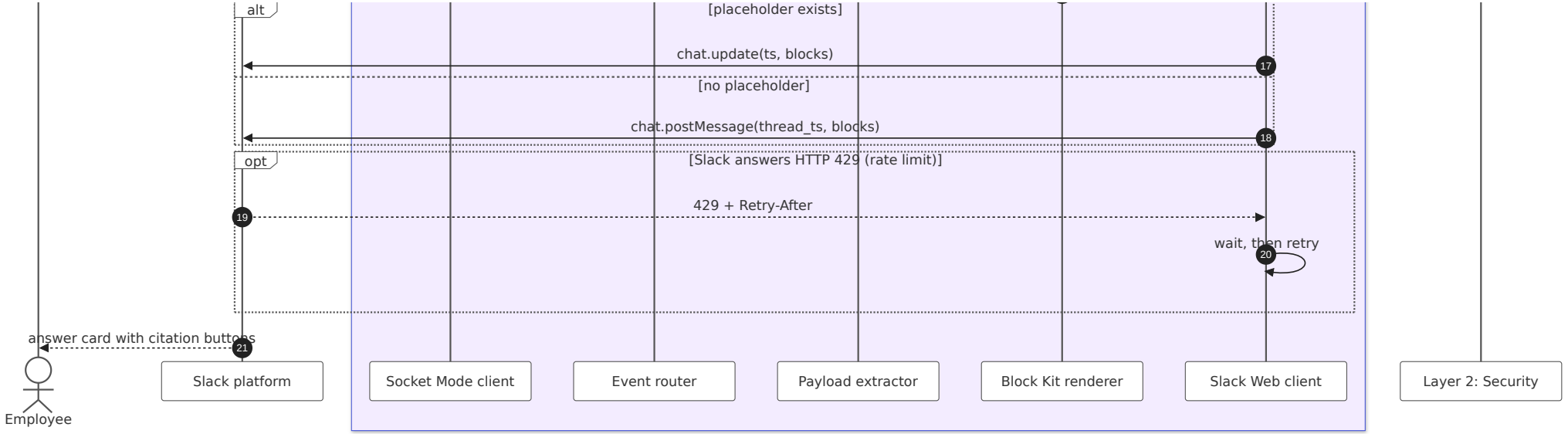


Diagram 3: Layer 1: Presentation (Slack app interface)

Layer 1

Purpose

Inside Layer 1: how the Bolt app receives a mention, turns it into a clean event payload, hands it to Layer 2, and later renders the result back into the same Slack thread. Layer 1 contains no business logic.

Participants

Participant	Role
Employee	Asks and reads the answer
Slack platform	Events and Web API (external)
Socket Mode client	Holds the WebSocket connection
Event router	Filters and dispatches events
Payload extractor	Builds the EventPayload
Block Kit renderer	Builds the answer card
Slack Web client	Posts and updates messages
Layer 2: Security	Next layer

Step by step

#	From → To	What happens	Why / note
Inbound: receive and prepare			
1	Employee → Slack platform	@bot question	
2	Slack platform → Socket Mode client	app_mention event over WSS	The bot opened the connection, so Slack only pushes events down it. The app-level token needs connections:write.
3	Socket Mode client → Event router	dispatch event	The client reconnects automatically if the connection drops.
4	Event router → Slack platform	↩ ack within 3 seconds	Sent first, before any slow work.
alt: event comes from the bot itself			
5	Event router ⟲	ignore	Prevents loops where the bot answers its own messages.
else: a real user mention			
6	Event router → Payload extractor	forward event	
7	Payload extractor → Slack platform	users.info(user_id)	Needs the users:read.email scope. The email is what Layer 2 uses to identify the person.
8	Slack platform → Payload extractor	↩ profile with email	
9	Payload extractor ⟲	build EventPayload	Fields: user_id, channel_id, thread_ts, text, email.
opt: placeholder message enabled			

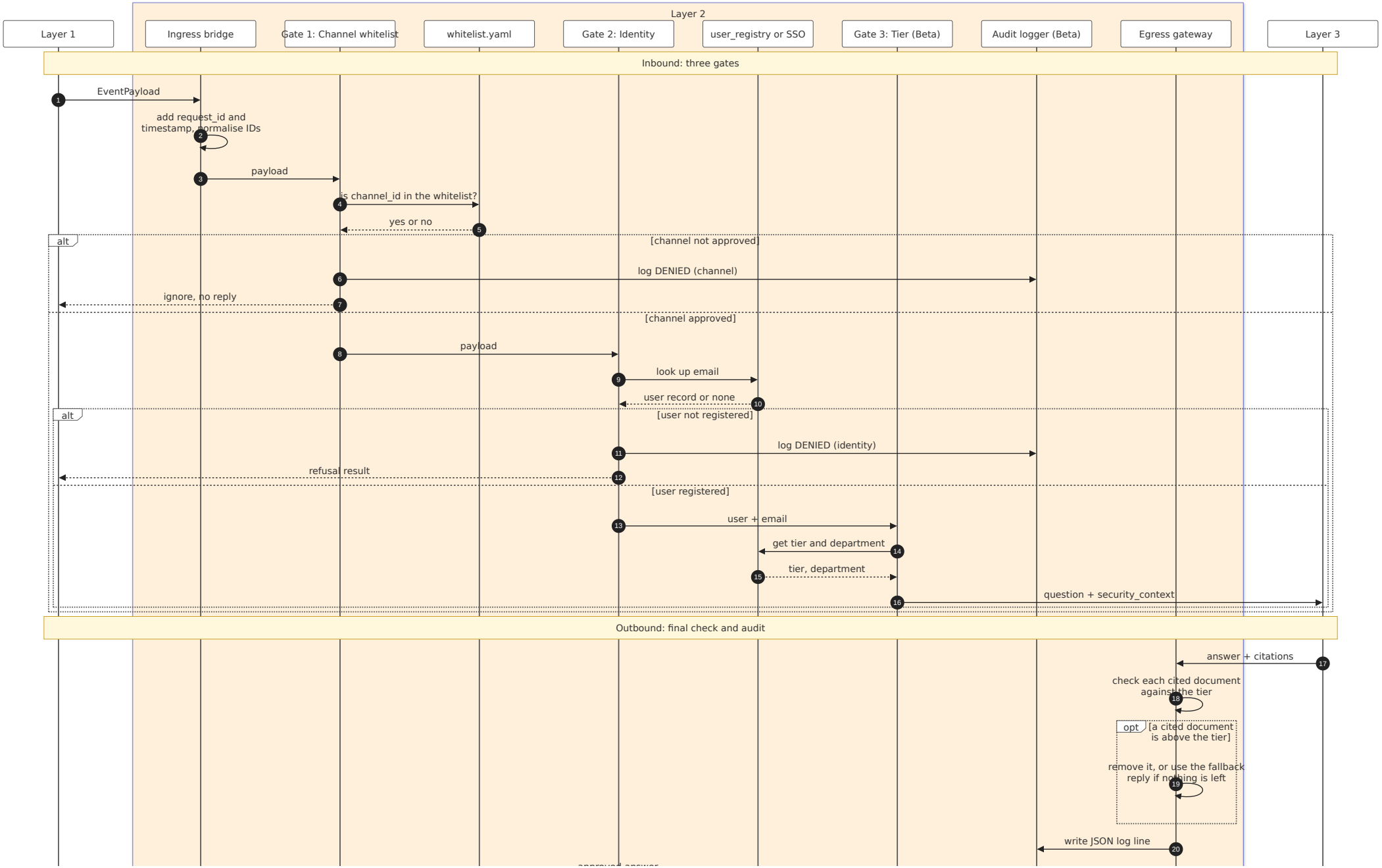
#	From → To	What happens	Why / note
10	Payload extractor → Slack Web client	request 'searching...' message	Gives the user instant feedback while the answer is prepared. (Design assumption.)
11	Slack Web client → Slack platform	chat.postMessage in thread	
12	Slack platform → Slack Web client	↩ message timestamp (ts)	The ts is needed later to update the same message.
13	Payload extractor → Layer 2: Security	EventPayload	From here Layer 1 only waits for the result.
Outbound: render and post			
14	Layer 2: Security → Block Kit renderer	↩ result: answer, fallback or denied	
15	Block Kit renderer ⟲	pick card variant and build blocks	Answer card: answer section, sources section, a button per source. Fallback and refusal cards are simpler.
16	Block Kit renderer → Slack Web client	blocks + thread_ts	
alt: placeholder exists			
17	Slack Web client → Slack platform	chat.update(ts, blocks)	Replaces the placeholder with the final card.
else: no placeholder			
18	Slack Web client → Slack platform	chat.postMessage(thread_ts, blocks)	Posts inside the thread to keep the channel tidy.
opt: Slack answers HTTP 429 (rate limit)			
19	Slack platform → Slack Web client	↩ 429 + Retry-After	
20	Slack Web client ⟲	wait, then retry	Back-off keeps the bot within Slack's limits.
21	Slack platform → Employee	↩ answer card with citation buttons	Each button opens the Confluence page the answer used.

Key points

- Slack requires an acknowledgement within 3 seconds. Heavy work runs after the ack, which is why the placeholder message helps the user.
- The bot ignores its own messages, which prevents answer loops.
- Reading the email needs the users:read.email scope. Layer 2 uses that email to identify the person.
- Slack roles (Owner, member) do not decide what the bot shows. The Confluence app installed in Slack is only for link previews and is not the data source.

Layer 2

Diagram 4: Layer 2: Security and authentication (1/2)



Layer 2

Diagram 4: Layer 2: Security and authentication (2/2)

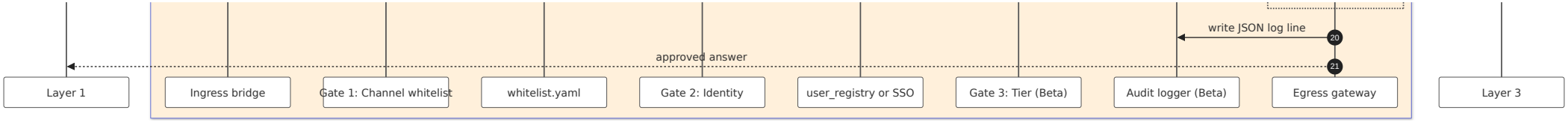


Diagram 4: Layer 2: Security and authentication

Layer 2

Purpose

Inside Layer 2: three gates in a fixed order (channel, identity, tier), then the final check, audit and hand-back on the way out. Deny by default: a failed gate stops the request and nothing reaches retrieval or the AI.

Participants

Participant	Role
Layer 1	Sends the payload, shows the result
Ingress bridge	Receives and normalises
Gate 1: Channel whitelist	Is this channel approved?
whitelist.yaml	Approved channel IDs (configuration)
Gate 2: Identity	Who is this person?
user_registry or SSO	Users, tiers, departments
Gate 3: Tier (Beta)	What may they see?
Audit logger (Beta)	One JSON log line per request
Egress gateway	Final check before the answer leaves
Layer 3	RAG core

Step by step

#	From → To	What happens	Why / note
Inbound: three gates			
1	Layer 1 → Ingress bridge	EventPayload	
2	Ingress bridge ⟲	add request_id and timestamp, normalise IDs	The request_id ties every log line of this request together.
3	Ingress bridge → Gate 1: Channel whitelist	payload	
4	Gate 1: Channel whitelist → whitelist.yaml	is channel_id in the whitelist?	Configuration lives in a file, not in code.
5	whitelist.yaml → Gate 1: Channel whitelist	↩ yes or no	
alt: channel not approved			
6	Gate 1: Channel whitelist → Audit logger (Beta)	log DENIED (channel)	
7	Gate 1: Channel whitelist → Layer 1	↩ ignore, no reply	The bot stays silent in channels it should not serve. (Design assumption.)
else: channel approved			
8	Gate 1: Channel whitelist → Gate 2: Identity	payload	
9	Gate 2: Identity → user_registry or SSO	look up email	Alpha: registry table approved by an admin. Project B: VIB SSO and the @vib.com.vn domain check.
10	user_registry or SSO → Gate 2: Identity	↩ user record or none	

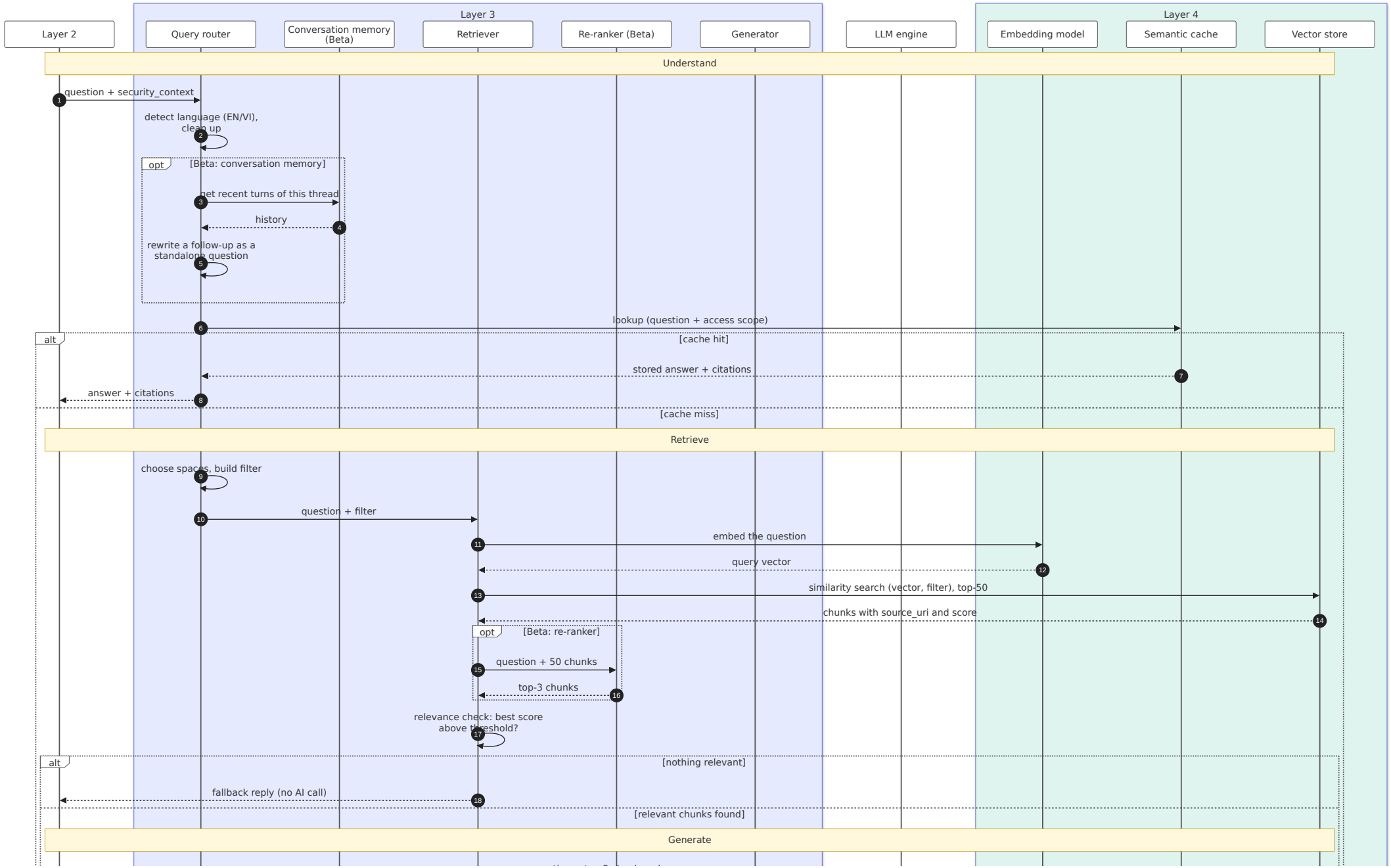
#	From → To	What happens	Why / note
alt: user not registered			
11	Gate 2: Identity → Audit logger (Beta)	log DENIED (identity)	
12	Gate 2: Identity → Layer 1	↩ refusal result	Layer 1 shows a short polite refusal.
else: user registered			
13	Gate 2: Identity → Gate 3: Tier (Beta)	user + email	
14	Gate 3: Tier (Beta) → user_registry or SSO	get tier and department	Tiers 1 to 4. Alpha uses tiers 1 and 2 only. (Proposal.)
15	user_registry or SSO → Gate 3: Tier (Beta)	↩ tier, department	
16	Gate 3: Tier (Beta) → Layer 3	question + security_context	security_context = user_id, email, tier, department, request_id.
Outbound: final check and audit			
17	Layer 3 → Egress gateway	answer + citations	
18	Egress gateway ⟲	check each cited document against the tier	Compares the document's access level and department with the asker.
opt: a cited document is above the tier			
19	Egress gateway ⟲	remove it, or use the fallback reply if nothing is left	
20	Egress gateway → Audit logger (Beta)	write JSON log line	Time, email, tier, query, document class, APPROVED or DENIED.
21	Egress gateway → Layer 1	↩ approved answer	

Key points

- The order is cheapest first: a wrong channel is rejected before any database lookup.
- Alpha identity is a registry table approved by an admin. Project B replaces that lookup with VIB single sign-on. The tier map stays the same.
- The output is a security_context. Layer 3 turns it into a database filter, so permissions are enforced before the AI sees any text.
- The outbound check is a second line of defence: even if retrieval were wrong, nothing above the asker's tier leaves Layer 2.

Layer 3

Diagram 5: Layer 3: RAG core and business logic (1/2)



Layer 3

Diagram 5: Layer 3: RAG core and business logic (2/2)

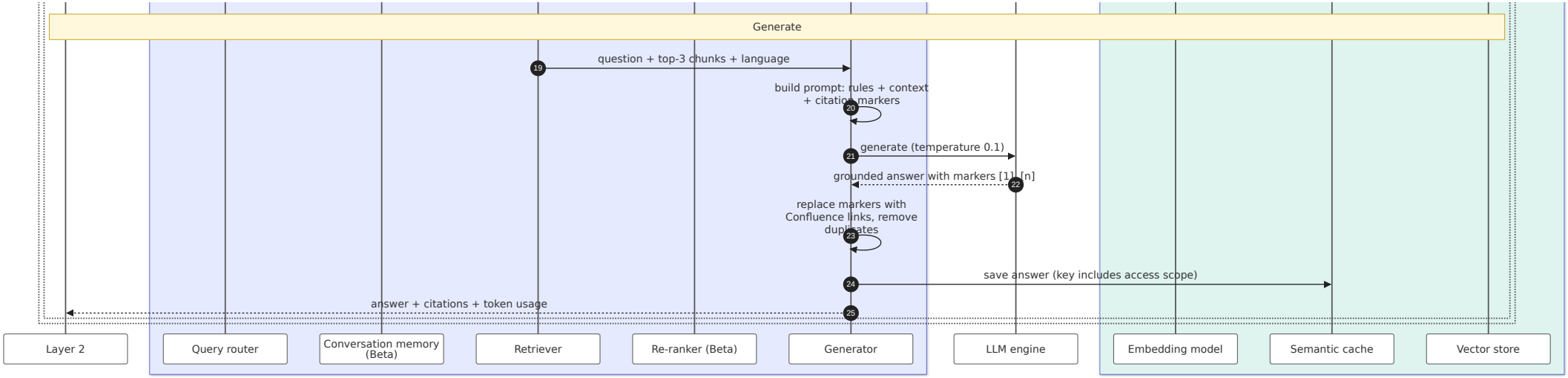


Diagram 5: Layer 3: RAG core and business logic

Layer 3

Purpose

Inside Layer 3: the question is understood, matched against the cache, turned into a filtered search, optionally re-ranked, and then either answered with sources or replied to with a safe fallback. Steps are grouped as understand, retrieve and generate.

Participants

Participant	Role
Layer 2	Sends the question and security_context
Query router	Language, filter, cache check
Conversation memory (Beta)	Recent turns of the thread
Retriever	Search and relevance check
Re-ranker (Beta)	Keeps the best three chunks
Generator	Prompt, citations
LLM engine	Ollama or Bedrock (adapter)
Embedding model	Turns the question into a vector
Semantic cache	Stored answers
Vector store	Filtered similarity search

Step by step

#	From → To	What happens	Why / note
Understand			
1	Layer 2 → Query router	question + security_context	
2	Query router ⟲	detect language (EN/VI), clean up	Removes the @mention and expands abbreviations. The language code travels with the request.
opt: Beta: conversation memory			
3	Query router → Conversation memory (Beta)	get recent turns of this thread	
4	Conversation memory (Beta) → Query router	↩ history	
5	Query router ⟲	rewrite a follow-up as a standalone question	For example 'and for sick leave?' becomes a complete question.
6	Query router → Semantic cache	lookup (question + access scope)	The scope is part of the key. (Design assumption.)
alt: cache hit			
7	Semantic cache → Query router	↩ stored answer + citations	
8	Query router → Layer 2	↩ answer + citations	The search and the AI call are skipped.
else: cache miss			
Retrieve			
9	Query router ⟲	choose spaces, build filter	Filter tokens: space_key, department_tag, access_level ≤ tier.

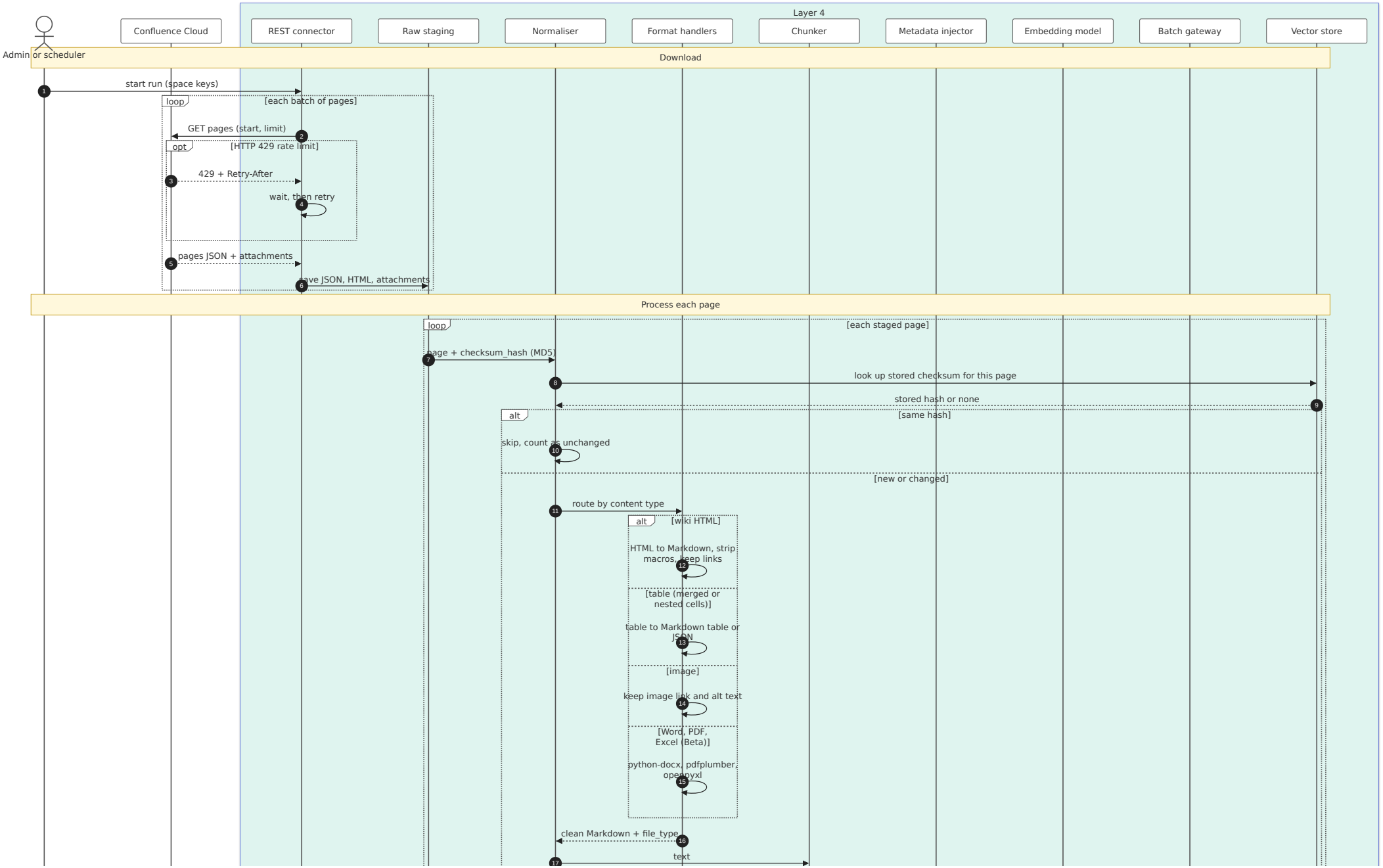
#	From → To	What happens	Why / note
10	Query router → Retriever	question + filter	
11	Retriever → Embedding model	embed the question	Must be the same model that embedded the documents.
12	Embedding model → Retriever	↩ query vector	
13	Retriever → Vector store	similarity search (vector, filter), top-50	Goes through the retrieval gateway (diagram 7).
14	Vector store → Retriever	↩ chunks with source_uri and score	
opt: Beta: re-ranker			
15	Retriever → Re-ranker (Beta)	question + 50 chunks	
16	Re-ranker (Beta) → Retriever	↩ top-3 chunks	A slower, more precise model re-scores the 50 candidates.
17	Retriever ⟲	relevance check: best score above threshold?	Without the re-ranker the three highest-scoring chunks are used.
alt: nothing relevant			
18	Retriever → Layer 2	↩ fallback reply (no AI call)	Said in the user's language. Not cached.
else: relevant chunks found			
Generate			
19	Retriever → Generator	question + top-3 chunks + language	
20	Generator ⟲	build prompt: rules + context + citation markers	Rules: use only the context, cite every claim as [1] to [n], answer in the question's language.
21	Generator → LLM engine	generate (temperature 0.1)	Low temperature keeps the answer stable and close to the documents.
22	LLM engine → Generator	↩ grounded answer with markers [1]..[n]	
23	Generator ⟲	replace markers with Confluence links, remove duplicates	source_uri becomes a clickable link, ordered by relevance.
24	Generator → Semantic cache	save answer (key includes access scope)	
25	Generator → Layer 2	↩ answer + citations + token usage	

Key points

- The filter is built from the security_context before any search runs, so the person's permissions shape the query itself.
- The cache key must include the user's access scope. Without it, an answer built for a manager could be shown to an employee. (Design assumption, needs team confirmation.)
- If no chunk scores above the threshold the layer replies with a fallback and makes no AI call, so it cannot invent an answer.
- The LLM, vector store, embedding model and source are used through adapters. Switching Ollama (Project A) to Bedrock (Project B) is a configuration change.
- Beta parts: conversation memory and the re-ranker. Without them the three highest-scoring chunks are used.

Layer 4

Diagram 6: Layer 4: Ingestion in detail (offline) (1/2)



Layer 4

Diagram 6: Layer 4: Ingestion in detail (offline) (2/2)

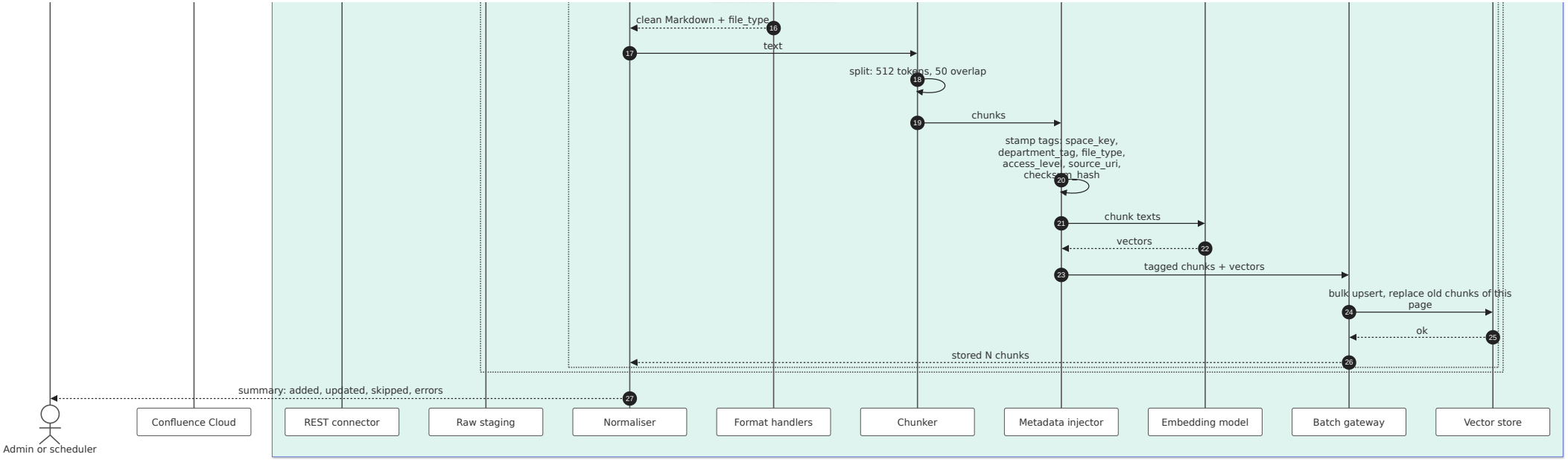


Diagram 6: Layer 4: Ingestion in detail (offline)

Layer 4

Purpose

The detailed version of diagram 2: how a page is downloaded with paging and retries, checked against its stored checksum, routed to the right format handler, cut into chunks, tagged, embedded and written to the store.

Participants

Participant	Role
Admin or scheduler	Starts a run
Confluence Cloud	Source (external)
REST connector	Pagination, retries
Raw staging	Local folder or S3
Normaliser	Checksum check, orchestration
Format handlers	One handler per content type
Chunker	512 tokens, 50 overlap
Metadata injector	Stamps the tags
Embedding model	Text to vector
Batch gateway	Bulk writes
Vector store	PostgreSQL + pgvector

Step by step

#	From → To	What happens	Why / note
Download			
1	Admin or scheduler → REST connector	start run (space keys)	
loop: each batch of pages			
2	REST connector → Confluence Cloud	GET pages (start, limit)	Uses the API token from an environment variable.
opt: HTTP 429 rate limit			
3	Confluence Cloud → REST connector	↩ 429 + Retry-After	
4	REST connector ⟲	wait, then retry	
5	Confluence Cloud → REST connector	↩ pages JSON + attachments	
6	REST connector → Raw staging	save JSON, HTML, attachments	Project A: local /data/raw_dumps. Project B: S3 with SSE-S3 encryption.
Process each page			
loop: each staged page			
7	Raw staging → Normaliser	page + checksum_hash (MD5)	
8	Normaliser → Vector store	look up stored checksum for this page	
9	Vector store → Normaliser	↩ stored hash or none	
alt: same hash			

#	From → To	What happens	Why / note
10	Normaliser ⟲	skip, count as unchanged	
else: new or changed			
11	Normaliser → Format handlers	route by content type	
alt: wiki HTML			
12	Format handlers ⟲	HTML to Markdown, strip macros, keep links	BeautifulSoup4.
else: table (merged or nested cells)			
13	Format handlers ⟲	table to Markdown table or JSON	Keeps numbers attached to their column headers. html5lib, pandas, Unstructured.
else: image			
14	Format handlers ⟲	keep image link and alt text	OCR of screenshots is Beta.
else: Word, PDF, Excel (Beta)			
15	Format handlers ⟲	python-docx, pdfplumber, openpyxl	Legacy .doc is converted with LibreOffice first.
16	Format handlers → Normaliser	↩ clean Markdown + file_type	
17	Normaliser → Chunker	text	
18	Chunker ⟲	split: 512 tokens, 50 overlap	The overlap keeps an idea from being lost at a cut.
19	Chunker → Metadata injector	chunks	
20	Metadata injector ⟲	stamp tags: space_key, department_tag, file_type, access_level, source_uri, checksum_hash	The most important step for security.
21	Metadata injector → Embedding model	chunk texts	
22	Embedding model → Metadata injector	↩ vectors	The vector size must match the database column.
23	Metadata injector → Batch gateway	tagged chunks + vectors	
24	Batch gateway → Vector store	bulk upsert, replace old chunks of this page	psycopg3 and SQLAlchemy bulk insert (COPY).
25	Vector store → Batch gateway	↩ ok	
26	Batch gateway → Normaliser	↩ stored N chunks	
27	Normaliser → Admin or scheduler	↩ summary: added, updated, skipped, errors	

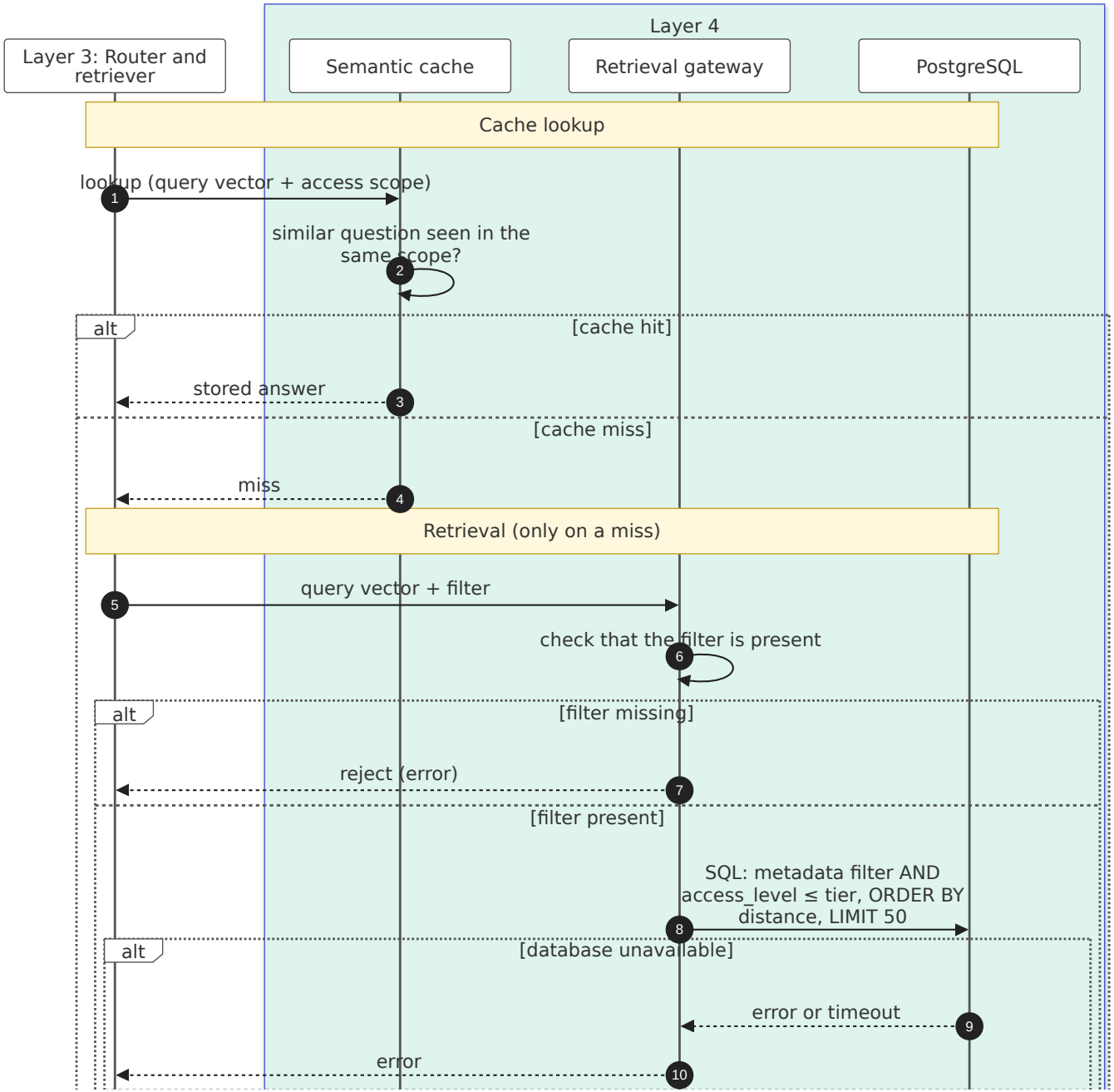
Key points

- Retry and raw staging protect Confluence from repeated calls while the team debugs the parsers.
- Each content type has its own handler. Alpha covers wiki pages, complex tables and images. Word, PDF, Excel and OCR are Beta.
- Tags are stamped before embedding, so every stored row carries its own access level.

- An edited page replaces its old chunks, otherwise old and new text would both be searchable. (Design assumption.)
- Gap to decide: a page deleted in Confluence leaves orphan chunks. A periodic check of page IDs can remove them. (Proposal.)

Layer 4

Diagram 7: Layer 4: Query-time retrieval and cache (1/2)



Layer 4

Diagram 7: Layer 4: Query-time retrieval and cache (2/2)

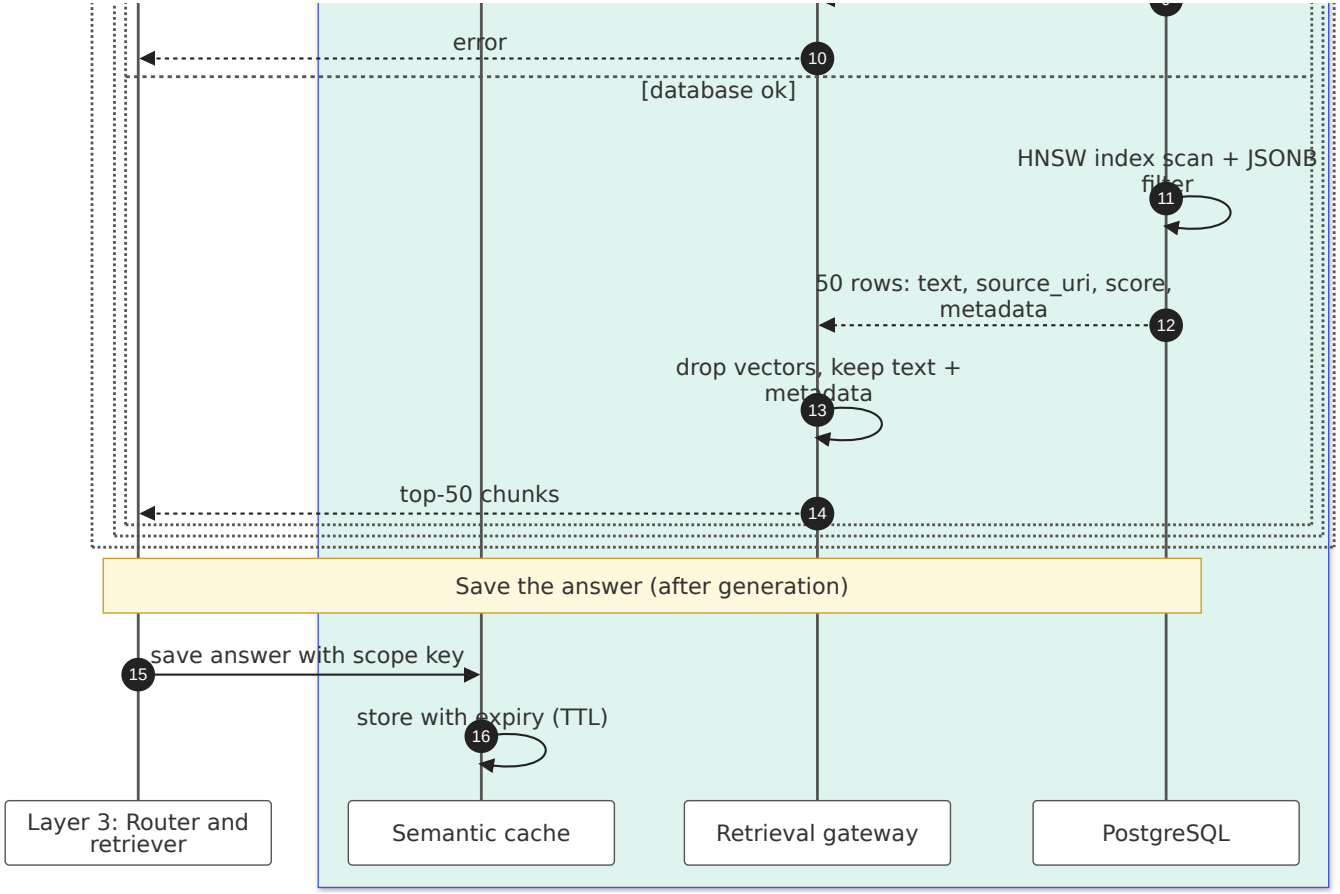


Diagram 7: Layer 4: Query-time retrieval and cache

Layer 4

Purpose

Inside Layer 4 when a question arrives: the cache is checked, the retrieval gateway enforces the access filter, PostgreSQL runs one query that combines the filter with vector similarity, and later the answer is saved back into the cache.

Participants

Participant	Role
Layer 3: Router and retriever	Asks and receives chunks
Semantic cache	Redis or in-memory dict
Retrieval gateway	Validates and runs the query
PostgreSQL	pgvector (HNSW) + JSONB metadata

Step by step

#	From → To	What happens	Why / note
Cache lookup			
1	Layer 3: Router and retriever → Semantic cache	lookup (query vector + access scope)	
2	Semantic cache ⟲	similar question seen in the same scope?	Compares the vector with stored ones within a similarity limit.
alt: cache hit			
3	Semantic cache → Layer 3: Router and retriever	↩ stored answer	Returns at once, so the response is very fast.
else: cache miss			
4	Semantic cache → Layer 3: Router and retriever	↩ miss	
Retrieval (only on a miss)			
5	Layer 3: Router and retriever → Retrieval gateway	query vector + filter	Filter: space_key, department_tag, access_level ≤ tier.
6	Retrieval gateway ⟲	check that the filter is present	A search without an access filter is refused.
alt: filter missing			
7	Retrieval gateway → Layer 3: Router and retriever	↩ reject (error)	Fail closed.
else: filter present			
8	Retrieval gateway → PostgreSQL	SQL: metadata filter AND access_level ≤ tier, ORDER BY distance, LIMIT 50	One query does both the permission filter and the similarity ranking.
alt: database unavailable			
9	PostgreSQL → Retrieval gateway	↩ error or timeout	
10	Retrieval gateway → Layer 3: Router and retriever	↩ error	Layer 3 shows a friendly message and the incident is logged.
else: database ok			

#	From → To	What happens	Why / note
11	PostgreSQL ⟲	HNSW index scan + JSONB filter	The index finds near neighbours quickly in milliseconds.
12	PostgreSQL → Retrieval gateway	↩ 50 rows: text, source_uri, score, metadata	
13	Retrieval gateway ⟲	drop vectors, keep text + metadata	Vectors are large and not needed by Layer 3.
14	Retrieval gateway → Layer 3: Router and retriever	↩ top-50 chunks	
Save the answer (after generation)			
15	Layer 3: Router and retriever → Semantic cache	save answer with scope key	Only answers that have sources are saved.
16	Semantic cache ⟲	store with expiry (TTL)	Expiry stops stale answers living forever after a page changes. (Design assumption.)

Key points

- Security and similarity are applied in one query because the vector, text and tags sit in the same row.
- The gateway fails closed: a search without a filter is never run. (Design assumption.)
- If the database is unavailable the gateway returns an error and Layer 3 shows a friendly 'try again' message instead of crashing.
- The team draft also describes caching search results. This diagram caches final answers, which also saves the AI call. Pick one approach in the decision record.

Design assumptions to confirm with the team

The diagrams include the following choices that are not stated in the Charter or the group's draw.io. They are proposals. Please confirm or change them before they become requirements.

1. Unknown channels are ignored silently. Refused users receive a short polite message.
2. The cache key includes the user's access scope (tier and department). This is needed to stop an answer for one tier appearing for another.
3. A short 'searching...' message appears at once and is replaced by the answer (Layer 1).
4. Slack rate limits (HTTP 429) are handled with a wait and retry.
5. The retrieval gateway never runs a search without an access filter (fail closed).
6. An edited Confluence page replaces its old chunks. Deleted pages are cleaned by a periodic check (proposal).
7. Cached answers expire after a time limit (TTL).
8. Tier mapping: project manager tier 4, developers tier 2, the client tier 1 (proposal). Alpha uses tiers 1 and 2.
9. Beta parts (tier check, audit log, re-ranker, conversation memory, Word/PDF/Excel/OCR handlers) are drawn so they can be switched off without breaking the Alpha flow.